

Πανεπιστήμιο Αιγαίου
Τμήμα Μηχανικών Πληροφοριακών και Επικοινωνιακών Συστημάτων

ICSD181 – Προγραμματισμός στο Διαδίκτυο

Διδάσκων: Κρητικός Κυριάκος

Car Workshop - Απαλλακτική

https://youtu.be/zfhrpRiR2mc

321/2022012 Δημήτριος Βασιλειάδης

Σάμος, Κυριακή 31 Αυγούστου, 2025

Κατάλογος Περιεχομένων

https://youtu.be/zfhrpRiR2mc **1**
Introduction **3**
Basic Architecture and Use-case **4**
Database Schema **6**
Authentication, Cookies and $_SESSION parameter **11**
Authentication 11
Cookies 13
$_SESSION 13
Security **15**
Session hardening and lifecycle 15
Cookie and header protections 15
Brute-force / rate limiting 15
Centralized sanitization 15
Password storage and verification 15
CSRF 16
XSS 16
SQL Injection 16
Debug mode and Logging **17**
Services and Utils **18**
src/services/ValidationService.php 18
src/services/FormProcessor.php 19
src/utils/Utils.php 20
Javascript and API **22**
FrontEnd **28**
Models **29**
How to set up **30**
The experience **30**

Introduction

The goal of this project is creating a web page for a car mechanic shop, with full BackEnd implementation. The system hosts three different types of Users: Secretary, Mechanic and Customer, with each having their own tools and utilities. The aim of this system was creating a web page with all functionality, while also utilising AJAX and API's in order to make the experience fluent for the user. Once the user has logged in and has accessed their dashboard, no other page reloads will happen, and the user is able to complete all actions within the dashboard. The aim was to create a SPA, as to experiment more with JavaScript.

Bootstrap 5 was used as the sole CSS library, with only one outstanding css file "custom.css" that primarily focused on Footer behavior.

Visual Studio Code was used as the Source Code Editor, XAMPP was used for both APACHE webservices and MySql using MariaDB. AI Tools were used for the creation of the database , the analysis and judgement of the code as per security, architecture and coding practices, and used for filling code blocks with Debug Statements when issues arose.

This "Report" is an attempt at documentation for a piece of software. I am not familiar with documenting with the intent of informing and educating a developer for maintenance or further improvement, so I consider this my first attempt.

Basic Architecture and Use-case

This application follows a MVC approach. The WorkSpace is split into /src and /public. The basic html views are located under /public/*.php, while they also use components split into /public/components and /public/layouts. Especially important is the DashboardComponents file, in which components reused by the dashboards are located.

The Backend, written in PHP, handles the core business logic and database interactions, with models in src/models encapsulating data access such as “User.php” hosting functions that make sense for the environment and services in src/services managing validation and business rules. Inside services, “FormProcessor” and “ValidationService” can be found which handle the processing of form submissions and validating input data accordingly.
Controlling actions are split between the API endpoints and Server-rendered pages. They either directly use ValidationService for validation or delegate orchestration to FormProcessor, which centralizes CSRF checks, sanitization, validation, model calls and DB transactions. Controllers then convert FormProcessor results into JSON responses (via sendSuccess / sendError) or render UI output for dashboards.

For the frontend, in order to keep true to the SPA, Bootstrap Modals are used for nearly all operations. Each user has functionality depending on their role, and can access different modals.

For example, use case: User presses Create Appointment, fills the form and presses submit. They following actions ensue.

When the user presses the submit button, function initAppointmentForm(formId, options = {}) in forms.js intercepts the submission. The attached listener calls e.preventDefault() to stop the default browser behavior, and immediately invokes handleFormSubmit(form, { endpoint: 'appointments.php', method: 'POST', ...options }).

Inside app.js, the function handleFormSubmit creates a FormData object from the form fields and calls fetchAPI(endpoint, options) to send the request.

The fetchAPI function builds the target URL by joining Config.apiBase with the endpoint. It prepares request headers (Accept: application/json, X-Requested-With: XMLHttpRequest), handles multipart form boundaries automatically if the payload is a FormData, and attaches CSRF protection both as a header (X-CSRF-Token) and as a hidden field inside the FormData. With this complete request, it calls fetch(url, finalOptions), forces the response to be parsed as JSON, and returns the parsed result.

The request arrives at appointments.php on the server. First, headers and body are read, and CSRF validity is checked with \SecurityConfig::validateCSRFToken(...). If the token is accepted the handlePost() function processes the POST data by validating required fields using validateRequired($_POST, $requiredFields) and sanitizing input before running ValidationService::validateAppointmentData($data). If validation errors exist, they are collected. Role-based access is checked next, and if permitted, an Appointment model is instantiated and its createAppointment($data) method is called.

Within Appointment.php, createAppointment opens a database transaction, double-checks required fields, and specifically verifies problem_description. It then ensures mechanic availability with checkMechanicAvailability($appointment_date, $appointment_time), which queries the database for existing appointments. If one or more mechanics are free, one is selected and the appointment is inserted. The transaction is committed and success data (appointment ID and assigned mechanic) is returned. If no mechanics are available, or a database error occurs, the transaction is rolled back and an error is raised.

The outcome is wrapped by public/api/common.php in either sendSuccess(...) or sendError(...), producing a JSON object of the form { success: true|false, message, data?, errors? }.

In the browser, fetchAPI resolves with this JSON and returns it to handleFormSubmit. If the success flag is true, handleFormSubmit shows a success message with showSuccess(response.message), resets the form, hides the modal, and calls the onSuccess callback, which refreshes the appointment table. If the success flag is false, handleFormSubmit checks for response.errors. When present, displayFieldErrors(form, response.errors) highlights the affected inputs. Finally, a general error alert is shown to the user.

Database Schema

The database consists of the following tables:

appointments

tasks

customers

mechanics

users

vehicles

failed_login_attempts

user_activity_logs

user_sessions

The last 3 activity logging tables are empty, as the logging has been transferred to In-Memory php. In a previous implementation, these 3 files where used as logging, now the logs exist in debug.

Users are split into 3 roles, yet only mechanics and customer have their own table as they hold more information than a simple user (AFM and specialty each).

Tasks are bound to an appointment.
The actual schema used for the creation of the database is the following:

```
-- 1. USERS TABLE (Χρήστης)

-- Base user table with authentication credentials and role assignment

CREATE TABLE users (

    id INT AUTO_INCREMENT PRIMARY KEY,

    username VARCHAR(50) NOT NULL UNIQUE,

    email VARCHAR(255) NOT NULL UNIQUE,

    password VARCHAR(255) NOT NULL,

    first_name VARCHAR(100) NOT NULL,

    last_name VARCHAR(100) NOT NULL,

    id_number VARCHAR(20) NOT NULL UNIQUE,

        role ENUM('visitor', 'customer', 'mechanic', 'secretary') NOT NULL DEFAULT 'visitor',

    status ENUM('inactive', 'active', 'suspended') NOT NULL DEFAULT 'inactive',

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,

    INDEX idx_email (email),
    INDEX idx_username (username),
    INDEX idx_role (role),
    INDEX idx_status (status),
    INDEX idx_id_number (id_number)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 2. CUSTOMERS TABLE
-- Extended information for customers - ΑΦΜ + address required
CREATE TABLE customers (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    afm VARCHAR(20) NOT NULL UNIQUE,
    address TEXT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_afm (afm),
    INDEX idx_user_id (user_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 3. MECHANICS TABLE
-- Extended information for mechanics - specialty required
CREATE TABLE mechanics (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
```

```
    specialty VARCHAR(200) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,

    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_user_id (user_id),
    INDEX idx_specialty (specialty)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 4. VEHICLES TABLE
-- Customer vehicles with detailed specifications - all fields required
CREATE TABLE vehicles (
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    serial_number VARCHAR(50) NOT NULL UNIQUE,
    model VARCHAR(100) NOT NULL,
    brand VARCHAR(100) NOT NULL,
    vehicle_type ENUM('επιβατικό', 'φορτηγό', 'λεωφορείο') NOT NULL,
        engine_type ENUM('ηλεκτροκίνητο', 'πετρελαιοκίνητο', 'υγραέριο', 'μεικτό') NOT NULL,
    door_count INT NOT NULL,
    wheel_count INT NOT NULL,
    production_date DATE NOT NULL,
    acquisition_year YEAR NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    deleted_at TIMESTAMP NULL,

        FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE,
    INDEX idx_customer_id (customer_id),
```

```
    INDEX idx_serial_number (serial_number),
    INDEX idx_brand_model (brand, model),
    INDEX idx_vehicle_type (vehicle_type),
    INDEX idx_engine_type (engine_type)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- 5. APPOINTMENTS TABLE
-- Customer appointments with state management
CREATE TABLE appointments (
    id INT AUTO_INCREMENT PRIMARY KEY,
    customer_id INT NOT NULL,
    vehicle_id INT NULL,
    mechanic_id INT NULL,
    appointment_date DATE NOT NULL,
    appointment_time TIME NOT NULL,
    reason ENUM('επιδιόρθωση', 'σέρβις') NOT NULL,
    problem_description TEXT NULL,
        status ENUM('ΔΗΜΙΟΥΡΓΗΜΕΝΟ', 'ΣΕ ΕΞΕΛΙΞΗ', 'ΠΕΡΑΙΩΜΕΝΟ', 'ΑΚΥΡΩΜΕΝΟ') NOT NULL DEFAULT 'ΔΗΜΙΟΥΡΓΗΜΕΝΟ',
    total_cost DECIMAL(10,2) NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
      updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    completed_at TIMESTAMP NULL,
    cancelled_at TIMESTAMP NULL,


      FOREIGN KEY (customer_id) REFERENCES customers(id) ON DELETE CASCADE,
    FOREIGN KEY (vehicle_id) REFERENCES vehicles(id) ON DELETE SET NULL,
    FOREIGN KEY (mechanic_id) REFERENCES mechanics(id) ON DELETE SET NULL,
    INDEX idx_customer_id (customer_id),
    INDEX idx_vehicle_id (vehicle_id),
```

```
    INDEX idx_mechanic_id (mechanic_id),
    INDEX idx_appointment_date (appointment_date),
    INDEX idx_status (status),
    INDEX idx_reason (reason),

    -- Constraint: problem_description required when reason is 'επιδιόρθωση'
    CONSTRAINT chk_problem_description
        CHECK (reason = 'σέρβις' OR (reason = 'επιδιόρθωση' AND problem_description
IS NOT NULL))
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;


-- 6. TASKS TABLE
-- Work tasks performed by mechanics on vehicles for appointments
CREATE TABLE tasks (
    id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL,
    mechanic_id INT NOT NULL,
    description TEXT NOT NULL,
    materials TEXT NOT NULL,
    completion_time INT NOT NULL,
    cost DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
     FOREIGN KEY (appointment_id) REFERENCES appointments(id) ON DELETE
CASCADE,
        FOREIGN KEY (mechanic_id) REFERENCES mechanics(id) ON DELETE
CASCADE,
    INDEX idx_appointment_id (appointment_id),
    INDEX idx_mechanic_id (mechanic_id),
    INDEX idx_cost (cost)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

Authentication, Cookies and $_SESSION parameter

Authentication

In this system, user identity and session state are managed through the AuthMiddleware within Auth.php file. The file auth.php holds the following function:

Starting with the class, AuthMiddleware is the application-level authentication gatekeeper implemented in auth.php.It's constructor calls secureSessionStart() so every request that includes the file ensures a properly configured PHP session. secureSessionStart() sets session cookie parameters (session name CAR_WORKSHOP_SESSION, lifetime 3600s, path /, SameSite=Lax, HttpOnly true, secure false in dev), enables strict session mode, starts the session, regenerates a new id when a session is first created, and updates $_SESSION['last_activity'] while enforcing idle timeout. This method centralizes session cookie config and idle-time enforcement so other pages can rely on consistent session behavior.

requireLogin($redirectTo) enforces authentication: if the user is not logged in it detects AJAX/JSON callers and returns a JSON envelope with HTTP 401 (after clearing output buffers), or for normal page requests it saves the intended URL to $_SESSION['intended_url'] and issues an HTTP redirect to the login page. requireRole($role) calls requireLogin and then checks the user role via hasRole, redirecting to an unauthorized page on failure. redirectIfLoggedIn($redirectTo) sends already-authenticated users away from public pages.

isLoggedIn() is a simple boolean check that returns true when $_SESSION['user_id'] exists and validateSession() passes. The private isAjaxRequest() helper inspects HTTP_X_REQUESTED_WITH, the Accept header for application/json, and CONTENT_TYPE to decide whether the caller expects JSON; this drives whether requireLogin returns JSON or redirects.

validateSession() enforces session lifetime and fingerprint checks. It ensures $_SESSION['created'] and $_SESSION['last_activity'] exist, rejects the session if idle timeout is exceeded, and if a fingerprint is present it computes the expected fingerprint and rejects the session on mismatch; it updates $_SESSION['last_activity'] when the session is valid. The fingerprint is a lightweight binding of a user and client context to the session and is computed by computeFingerprint($userId).

loginUser($email, $password) is the central authentication routine. It first calls isLockedOut($email) to check per-session lockout counters. It queries the users table for the supplied email (ignoring deleted rows), and if no row is found or password_verify($password, $user['password']) fails it calls recordFailedLogin($email) (which increments counters in $_SESSION) and returns an error. If the record exists but status !== 'active' the function returns an appropriate message. On successful verification it calls createUserSession($user), clearFailedLogins($email), and returns success plus a redirect URL based on role via getRedirectUrl($role). Note: authentication uses PHP's password_verify() to compare the typed password with the stored hash . The PlainText password is not stored, only the hash version of it.

createUserSession(array $user) establishes the server-side session after successful login. It regenerates the session id (to prevent fixation), writes $_SESSION['user_id'], $_SESSION['email'], $_SESSION['username'], $_SESSION['role'], sets $_SESSION['last_activity'] and $_SESSION['created'] timestamps, and sets $_SESSION['fingerprint'] = $this->computeFingerprint($userId). computeFingerprint(int $userId) builds a short binding (the code hashes the User-Agent with the user id using hash('sha256', $userAgent . '|' . $userId)) so the server can detect obvious session re-use on a different client. Important operational detail: createUserSession only updates $_SESSION; it does not insert a row in the user_sessions DB table (so that table remains empty unless you add DB writes).

logout() calls destroySession(), which clears $_SESSION, unsets the session cookie, and calls session_destroy() so the server-side session data is removed. destroySession() also clears the cookie by setting its expiry in the past.

hasRole(string $requiredRole) looks up $_SESSION['role'] and uses AppConfig::hasPermission to decide whether the current user meets role requirements. getCurrentUser() returns a small user info array built from $_SESSION and attempts to lazy-load extra fields (first_name, last_name, id_number) from the users table using the user_id from session.

Lockout and brute-force protections are implemented in-session: isLockedOut($email) reads $_SESSION['login_attempts'][$email] and $_SESSION['last_attempt'][$email] to enforce MAX_LOGIN_ATTEMPTS and LOCKOUT_TIME. recordFailedLogin($email) increments the per-session counter and sets the last attempt timestamp; clearFailedLogins removes these counters. There is also another lightweight throttling implementation in login_process.php that uses $_SESSION['last_login_attempt'] and $_SESSION['login_attempt_count'] to throttle rapid repeated attempts.

Cookies

Between the server and the client only 2 cookies are communicated, 1) CAR_WORKSHOP_SESSION (set by session_name('CAR_WORKSHOP_SESSION') in auth.php) and language set by language.php with setcookie('language', ..., time() + 30*24*60*60, '/').

The language cookie is not HttpOnly and can be read by client-side scripts.

When sessions are active, LanguageManager also writes the same value into $_SESSION['language'].

While not a cookie, a CSRF token is also passed with each state changing request, generated by the server and saved onto $_SESSION['csrf_token'], and shown on each dashboard.

$_SESSION

The $_SESSION superglobal in PHP is the storage for session data. It is created when a session starts and is keyed by the session ID, which the browser sends in the CAR_WORKSHOP_SESSION cookie. The contents of $_SESSION remain on the server and are looked up whenever a request comes in with a valid session ID.

This application saves the following information

csrf_token — string: per-session CSRF secret generated by src/config/AppConfig.php::generateCSRFToken() and used to validate mutating requests (set/read in AppConfig.php, rendered into pages via templates like public/*_dashboard.php, consumed by API endpoints such as appointments.php).

user_id — int: authenticated user users.id, set on login in includes/auth.php::createUserSession() and read across auth and API code (auth.php, common.php, various controllers).

email — string: authenticated user email, set in createUserSession() and read in includes/auth.php::getCurrentUser() and other places.

username — string: authenticated username, set in createUserSession() and used by UI code/pages.

role — string: user role (visitor|customer|mechanic|secretary), set in createUserSession() and used for authorization checks (e.g. includes/auth.php::hasRole()).

fingerprint — string: session fingerprint (hash of User-Agent + user id) set in createUserSession() and validated in validateSession() to detect obvious session hijack.

created — int (timestamp): session creation time, set in includes/auth.php::secureSessionStart() / createUserSession() and used for session lifecycle.

last_activity — int (timestamp): last request time, updated in secureSessionStart() / validateSession() to enforce idle timeouts.

intended_url — string: stored by requireLogin() to remember where to redirect after successful login.

flash_message — string or array: transient UI message stored by Utils::redirectWithMessage() or and read/cleared by Utils::getFlashMessage()

flash_type — string: type for the flash message (e.g., success, info), set along with flash_message.

language — string: selected language code (e.g., en, el), set/read by includes/language.php::LanguageManager (copied from URL param, cookie, or $_COOKIE['language'] into $_SESSION['language']).

login_attempts — array: per-email counters used by includes/auth.php::isLockedOut() and recordFailedLogin() (shape: ['email@example.com' => int]).

last_attempt — array: per-email unix timestamp of last failed attempt (used with login_attempts).

last_login_attempt — int: timestamp used by login_process.php to throttle rapid attempts (separate lightweight throttle).

login_attempt_count — int: short-term count used by login_process.php (paired with last_login_attempt).

rate_limits — array: per-session rate-limiting state used by register.php (shape and keys managed inside that file).

Security

Session hardening and lifecycle

AuthMiddleware starts sessions with session_set_cookie_params (httponly, samesite=Lax), session.use_strict_mode=1, regenerates session id on creation/login, enforces idle timeout, stores lightweight fingerprint (User-Agent + user id), and destroys session on mismatch/timeouts. The rules arent strict as HTTPS is not available and its currently in development mode, but the variables can be changed at any time.

Cookie and header protections

Responses set security headers: X-Content-Type-Options: nosniff (revents browsers from MIME-sniffing responses and executing them as a different type (helps prevent certain XSS vectors where an attacker supplies a payload with a confusing content type) , X-Frame-Options: DENY (prevents the page from being embedded in any iframe, blocking classic clickjacking vectors) X-XSS-Protection: 1(for old browsers); mode=block (e.g., login_process.php) and JSON endpoints set content-type & cache-control (common.php) (stops intermediate caches (and browsers) from storing sensitive API responses, reducing the risk that protected JSON would be served to unauthorized users later)

Brute-force / rate limiting

Login lockout counters and lockout window in AuthMiddleware (MAX_LOGIN_ATTEMPTS, LOCKOUT_TIME), per-session rate limiting/time-checks in login_process.php. MAX_LOGIN_ATTEMPTS are at 5 and LOCKOUT_TIME at 1 hour from last_activity

Centralized sanitization

ValidationService.php performs server-side validation (regexes, enums, ranges, date checks). ValidationService::sanitizeInput trims values; common.php also offers sanitizeInput and getJsonInput.

Role-based access control

AuthMiddleware::requireRole() and AppConfig::hasPermission() used to enforce page-level role restrictions. On top of every role protected file there exists the following lines

```
$auth->requireLogin();

$currentUser = $auth->getCurrentUser();

$currentRole = $currentUser['role'];
```

Password storage and verification

The database stores the password hash. During login the code calls password_verify($typedPassword, $storedHash) which returns true only when the typed

password matches the stored hash. The application never stores the plaintext password in the DB or in the session; only the hash is persisted. Therefore the server cannot recover the original plain password from the DB, incase an attack happens that same password cannot be recovered from the attacker.

CSRF

The app issues and validates CSRF tokens and ties them to the session. Tokens are generated/validated via SecurityConfig::generateCSRFToken() / SecurityConfig::validateCSRFToken() and embedded into server-rendered forms with Utils::csrfField(); server-side validation is performed (for example in login_process.php) and some FormProcessor flows check the token when supplied. The pages expose the current token for AJAX (window.csrfToken in public/mechanic_dashboard.php) so client scripts can include it in fetch/FormData requests. Some handlers only do “optional” check for csrf, FIX BEFORE RELEASE.

CSRF tokens stop CSRF attacks but do not protect against XSS. Rendering the token into window.csrfToken or any client-visible value makes it available to page scripts, so if the site suffers an XSS vulnerability an attacker who runs script in the page can read the token and then perform CSRF (or any action) as the user.

XSS

Output encoding and client-side escaping are used. Server-side rendering uses Utils::escape() (which calls htmlspecialchars and FormBuilder helpers to escape values inserted into HTML, and API input helpers (public/api/common.php::sanitizeInput) trim/escape inputs early. On the client forms.js provides escapeHtml() and many DOM updates use .textContent or explicit escaping when inserting JSON into the page (table rows, import summaries, modal fields). Defensive headers are also set in critical endpoints (X-Content-Type-Options: nosniff, X-Frame-Options: DENY, and X-XSS-Protection in login_process.php).

SQL Injection

database.php uses PDO with ATTR_EMULATE_PREPARES => false, while BaseModel and other models who query sql use prepared statements and parameter binding for queries.

Debug mode and Logging

Inside AppConfig there is a function called isDebugMode(). When AppConfig::isDebugMode() is true the application enables diagnostics: src/config/debug.php turns on full error reporting and display (error_reporting/display_errors), sets error_log to /Applications/XAMPP/xamppfiles/logs/php_debug.log, and exposes helper functions that write timestamped, print_r-style entries into the configured PHP error log. debug_request() is auto-run when the file is included in debug mode, so each request can record headers, $_GET/$_POST, and session data.

On the client, window.__APP_DEBUG__ gates console and AJAX logging: when true fetchAPI and many modules emit grouped request/response logs and raw response text; when false app.js mutes console methods. Server-side endpoints (some API helpers and the login flow) may also include debug_info in JSON responses when debug is enabled, making debug payloads visible to the browser. In short: server logs are written to the php_debug.log file (and PHP error stream) via error_log(). Client logs appear only in the browser console when window.__APP_DEBUG__ is true.

Services and Utils

FormProccessor, ValidationService and Utils.php provide many tools for most of the application functionality.

src/services/ValidationService.php

Centralized server-side validation and sanitization for incoming form and payload data.

sanitizeInput($input)

What: Recursively trims strings, normalizes arrays, removes extra whitespace.
Input: Scalar/string or array (typically `$_POST` or decoded JSON).
Output: Sanitized value (same shape).
Use: Accepting any user input — run first to normalize before validation.
Edge: Keeps numeric strings as strings; does not HTML-escape (escaping is separate).

validateRegistrationData(array $data)

What: Validates registration fields (username, email, password, role, AFM/id, etc).
Input: Sanitized registration payload.
Output: Array of errors (field => messages) or empty array on success.
Use: Registration endpoint or server-side registration flow.
Edge: Enforces role-specific checks (customer vs mechanic).

validateUserData(array $data)

What: Validates user create/update payload used by admin/secretary flows.
Use: Creating or editing users via admin UI.
Notes: Checks username/email uniqueness, formats, role requirements.

validateProfileData(array $data)

What: Validates profile update payload for the current user.
Use: User profile edits.

validateVehicleData(array $data)

What: Validates vehicle payload (plate, make/model, year, VIN).
Use: Vehicle creation or update.

validateAppointmentData(array $data)

What: Validates appointment payload (date/time, customer ID, reason, vehicle).
Use: Creating or updating appointments; checks date consistency.

validateTaskData(array $data)

What: Validates mechanic tasks (description, cost, appointment linkage).
Use: Creating or updating tasks.

formatErrors(array $errors)

What: Normalizes the error structure to the API's expected format (strings, arrays).
Use: Returning errors to the client.

hasErrors(array $errors)

What: Boolean check whether validation produced any error entries.
Use: Determine if validation produced errors.

---

src/services/FormProcessor.php

Central place to handle business logic for form submissions. Ensures transactions, permission checks, and model calls are applied consistently.

processRegistration(array $data)

What: Validates registration, creates User record, may send confirmation.
Use: `api/register.php` or registration controllers.
Behavior: Starts DB transaction, uses ValidationService, hashes passwords, returns structured response.

processProfileUpdate(int $userId, array $data)

What: Validates and applies profile updates, enforces ownership, commits transaction.
Use: Profile edit endpoints (`user-dashboard/profile`).

processVehicleCreation(int $userId, array $data)

What: Validates vehicle, creates vehicle record attached to owner, returns new record ID or errors.

processVehicleUpdate(int $userId, int $vehicleId, array $data)

What: Checks ownership, validates, updates record.

processTaskCreate(int $mechanicId, array $data)

What: Creates mechanic tasks, attaches to appointment, enforces permissions.

processTaskUpdate / processTaskDelete(...)

What: Updates or removes tasks with permission checks and transactional safety.

processUserCreate / processUserUpdate / processUserDelete

What: Admin user management flows; validates, enforces role/permission constraints, logs activity.

---

src/utils/Utils.php

Small, reusable helpers for escaping, CSRF, UI rendering, logging, and client detection.

escape($value)

What: HTML-escape (uses `htmlspecialchars` with `ENT_QUOTES`, UTF-8).
Input: String (or array if implemented).
Output: Escaped string safe for HTML output.
Use: Echoing user-supplied data into templates or attributes.

csrfField()

What: Returns an HTML hidden input containing a generated CSRF token.
Use: Rendering server-side forms — include inside `<form>` to ensure forms have token.

randomString($length)

What: Cryptographically suitable random string generator.
Use: Generating nonces or temporary tokens.

statusBadge($status)

What: Returns an HTML snippet or class for a status badge (escaped).
Use: Rendering status in dashboards.

FormBuilder Helpers (selectOptions, input, textarea)

What: Produce escaped form inputs/options.
Use: Building server-side forms; handle labels/options and selected/value attributes.

isAjaxRequest()

What: Heuristic to detect XHR/fetch requests.
Use: Endpoint logic that behaves differently for AJAX vs full-page requests.

getClientIP()

What: Returns best-effort client IP considering headers.
Use: Logging, rate-limiting, or geo checks.

logActivity($message, $data = null)

What: High-level activity logger.
Use: Record important user actions; wraps debug or audit logs.

dd($value)

What: Debug dump + die; prints value only in debug mode.
Use: Development only.

CSRF Helpers & JS Exposure

What: `csrfField()` is for server-side forms; utilities may inject `window.csrfToken` into templates.
Use: Ensure AJAX calls can read `window.csrfToken` and client fetch wrapper attaches it.

Javascript and API

This application uses JavaScript for client-side UX and to ensure a functionality similar to a SPA. There are 7 JavaScript files.

app.js , dashboard-common.js and forms.js are provided for central rules and reusability, while each dashboard and the registration has its own script bound to the UI.

The API calls are made using fetch(), using the fetchAPI() wrapper i created for csrf and formdata - JSON handling

The applicaition uses FormData when submitting HTML forms (js/forms.js) because FormData maps directly to browser forms and supports file uploads and multi-part fields. When a form is serialized with new FormData(form) the browser populates both text fields and file inputs, fetchAPI detects options.data instanceof FormData, removes the manual Content-Type so the browser will set the multipart boundary, and appends a csrf_token field into the FormData so the server can validate CSRF from $_POST.

JSON is used for API calls, AJAX that sends plain objects, or when the caller passes a JS object to fetchAPI. For these, fetchAPI stringifies the object and sets Content-Type: application/json. On the server side the API reads JSON (via helpers like getJsonInput())

app.js

The application core of the javascript files, provides the fetchAPI, handleFormSubmit and core application utilities and initialization. It hosts many small functions such as showError and showSuccess.
fetchAPI is a wrapper around the browser fetch that stardardizes how AJAX calls are made and how errors are handled. On statechanging requests(POST,PUT,DELETE), it tries to attach a CSRF toker

dashboard-common.js

Provides shared utilities for dashboards across roles such as cancelAppointment() and deleteVehicle(). The functions adapt to the user roles and allow for different actions,

forms.js

The central form/table/rendering engine: every CRUD interaction and table comes through this file. It wires forms (initAppointmentForm, initVehicleForm, initUserForm), runs CSV imports (handleCsvImport), and provides table refreshers (refreshAppointmentsTable, refreshVehiclesTable, refreshUsersTable). Hosts rendering and modal

detail functions (updateAppointmentsTable, updateVehiclesTable, updateUsersTable, viewAppointment, viewVehicle) plus task management (loadTasksManagement, editTask, deleteTask) and pagination (updatePagination, changePage).

Each dashboard has their own script with their own implementation of functions. Each dashboard has a init fuction that sets up the respective dashboard, search and filtering functions and role specific helpers.

All API endpoints route requests based on HTTP methods (GET,POST,PUT,DELETE) and responses are standardized using sendSuccess and sendError

appointments.php

- Methods: GET, POST, PUT, DELETE

GET

- Query params:
- `customer_appointments=1` — appointments for the current customer (filters: `search`, `status`, `date_from`, `date_to`, `page`, `per_page`)
- `mechanic_appointments=1` — mechanic's appointments (same filters)
- `secretary_appointments=1` — secretary view (same filters)
- `id` — single appointment by id
    - `action=available_slots&date=YYYY-MM-DD` — available time slots for date

POST (create)

- Payload: FormData or POST fields
- Required fields: `customer_id`, `vehicle_id`, `appointment_date` (YYYY-MM-DD), `appointment_time` (HH:MM), `reason`
- Roles: `customer` (for own appointments), `secretary` (can create for others)

PUT (update)

- Payload: JSON body (read with `getJsonInput()`)
- Required: `appointment_id`
- `action` sub-values (in JSON):
    - `update_status` — requires `status`
    - `update_datetime` — requires `appointment_date`, `appointment_time`
    - `cancel` — cancel the appointment
- Role checks apply: customers/mechanics/secretaries have different permissions

DELETE

- Query param: `id` (appointment id)
- Cancels the appointment (status/role checks enforced)

---

users.php

- Methods: GET, POST, PUT, DELETE

GET

- `action=get_profile` — returns current user's profile (any authenticated user)
- `id` — specific user (secretary only)
- Pagination: `page`, `per_page`, `q` (search) — secretary only

POST

- `action=update_profile` (POST fields) — update logged-in user's profile
- Else: create new user (secretary only). Required: `username`, `email`, `password`, `first_name`, `last_name`, `id_number`, `role`

PUT

- PUT body parsed from `php://input` (urlencoded/JSON depending on caller)
- `user_id` required
- Secretaries may toggle `status` / `new_status`
- Uses FormProcessor for validation/persistence

DELETE

- `user_id` required (parsed from body)
- Permission: secretary or same user

---

tasks.php

- Methods: GET, POST, PUT, DELETE

GET

- `task_id` — single task

- `appointment_id` — tasks for an appointment
- Else returns tasks scoped by role (mechanic assigned tasks or all for secretary)

POST (create)

- Payload: POST form data (FormProcessor)
- Required: `appointment_id`, `description`, `materials`, `completion_time` (minutes)
- Roles allowed: `mechanic`, `secretary`

PUT (update)

- Parsed from `php://input` (urlencoded)
- `task_id` required
- Role checks: mechanic for own tasks or secretary

DELETE

- `task_id` via body or `?task_id=` query
- Role checks enforced

---

vehicles.php

- Methods: GET, POST, PUT, DELETE (also supports form-based POST handlers)

GET

- `customer_vehicles=1` — vehicles for logged-in customer
- `id` — single vehicle

- `customer_id` — vehicles for a customer
- Paginated listing: `q` / `search`, `page`, `per_page`

POST (create)

- Accepts JSON or FormData
- Required: `customer_id`, `brand`, `model`, `serial_number`, `vehicle_type`, `engine_type`, `door_count`, `wheel_count`, `production_date`, `acquisition_year`
- Customers may only add vehicles to their own account

PUT (update)

- JSON body expected (also supported via form POST helper)
- `vehicle_id` required plus fields to update
- Permission: secretary or owning customer

DELETE

- JSON body: `id` required (or form `vehicle_id`)
- Permission: secretary or owning customer

---

check-availability.php

- Method: GET

GET

- Query params supported:
    - `date=YYYY-MM-DD` — returns available appointment slots for date

- `username` — username availability
- `email` — email availability

- Returns JSON with `available: true/false` and messages

---

register.php

- Method: POST (Form data)

POST

- Payload: registration form fields (username, email, password, role, etc.) plus `csrf_token`
- CSRF and session-based rate-limiting enforced

FrontEnd

The base goal of this application was to create a SPA experience. For this goal, a significant portion of the UI functionality relies on Bootstrap modals, which are used extensively for user interactions such as form submissions, appointment bookings, profile editing, and task management to name a few. After logging in, the user will not face a page reload on the dashboard – all actions are called through AJAX. The tables are refreshed using API calls (refreshTable() depending on the type of table) and the modals are dynamically rendered and populated with data using JavaScript, ensuring seamless integration with the backend APIs. Shared components like DashboardComponents.php provide reusable modal templates, such as the appointment and profile modals, which are customized for different roles (e.g., customer, mechanic, secretary). The reliance on Bootstrap modals ensures consistency in design and behavior across the application, while JavaScript utilities like forms.js and dashboard-common.js handle the dynamic content updates and AJAX-based workflows within these modals. The header and the footer are loaded throughout the application, and the dashboard has a sleek look shared through all three dashboards.

For example, the secretary has access to all the users. Each user row in the table of users has actions, one of those actions being to open the information of the user. The way this is implemented is with a generic modal (actually 1 generic modal with 2 mutations

depending on role) being rendered to the DOM. Opening this modal creates an API request with ID of the user the action was pressed for. The API request fills the modal with the text, and the secretary can look at the information of the user. Most of the actions are implemented in a similar fashion.

Models

All SQL queries are done inside the models, and handle the respective fields of each table of the database

BaseModel.php serves as the foundational class for all other models, providing shared database connection and CRUD operation methods. It simplifies repetitive database tasks by offering reusable query execution and error-handling utilities.

User.php handles authentication, role-based access control, and user-specific database operations.

Car.php provides methods for managing vehicle records, including creation, updates, and retrieval.

Appointment.php includes methods for handling appointment workflows, such as scheduling, updating statuses, and adding notes.

Task.php provides methods for managing task workflows, such as creating, updating, and completing tasks.

How to set up

In the folder a .sql file can be found. By importing this file onto MySQL you can have access to the database. All users currently in the database have the password = “password”. Here are a few login credentials to try, once the database has been imported.

admin@carworkshop.gr

customer1@example.com

customer2@example.com

mechanic1@carworkshop.gr

mechanic2@carworkshop.gr

Afterwards, simply use XAMPP and add the folder under “htdocs”.The navigation inside the application should be straight forward. If using XAMPP, then the URL should be:

“http://localhost/Apallaktiki/car-workshop-system/public/index.php”

Launching a PHP server through the IDE of your choice should also be feasible. I have a tasks.json for opening a server through php although i am unsure how it works outside of VScode.

The experience

Learning Javascript and php was not a pleasant experience. It was tedious, and the lack of experience led me to making bad decisions and ultimately scratching the (working) implementation of the application at the time and starting anew. I did not enjoy a single moment of coding this application and I was miserable throughout. However, towards the end of the coding, I started to understand what the guides and documentation wanted to teach me, and I understood that the world of Web Programming is quite large. A part of me regrets the decision to use php and javascript for this application, as using newer tools would have been more enjoyable. However I chose this architecture because I wanted to see what Web Programming entailed, and I consider learning Javascript to be valuable. I do not think I have a deep understanding of it, but I now understand at least the basics, enough to create a working AJAX application. This application, from start to finish, took me about 30 days (Including learning Javascript and php, finding out that i understood it wrong, relearning and scratching the implementation, and still not being content with the final product). This is because debugging took me a lot longer than expected for every single bug that I found, because many times I simply did not understand why things were not working. A big part of this difficulty was the dissonance of not having a compiler in my IDE, and having to search through logs and the Development Tools provided by the browser. By the time I learned what an .env is, it was way, way too late.